

Grafito CANStepper C3

CANopen Electronic Data Sheet (EDS) Configuration Guide

Beginner to advanced | Firmware 2.0 (CiA 301 + CiA 402 subset)

File: GrafitoCANStepper.eds | EDS version 4.0 | 13 September 2026

Grafito Innovations

This guide explains the whole EDS setup for the Grafito CAN stepper: what an EDS is, how to import it into a PLC, every object in the dictionary, and how to command motion like a drive profile. You do not need prior CANopen experience to start. Later chapters assume you have read the earlier ones.

Two firmwares, two protocols

The production Python product still uses GCSP v1 (GrafitoCANStepper_C3.ino, firmware 1.9). PLC / EDS support is the separate CANopen sketch (GrafitoCANStepper_C3_CANopen, firmware 2.0). Do not mix GCSP and CANopen nodes on the same CAN bus. Flash CANopen firmware before importing this EDS.

Contents

1. Part 1 — Beginner: what you are configuring
2. Part 2 — Intermediate: the EDS file and object dictionary
3. Part 3 — Intermediate: CiA 402 drive profile
4. Part 4 — Advanced: PDOs, COB-IDs, SDO and bus timing
5. Part 5 — Advanced: PLC import and first-motion recipes
6. Part 6 — Reference tables and troubleshooting

Part 1 — Beginner: what you are configuring **BEGINNER**

1.1 The hardware in one paragraph

A Grafito CANStepper C3 board is a small CAN node: ESP32-C3 microcontroller, TMC2209 stepper driver, MT6701 magnetic encoder (16384 counts per revolution), and a TCAN3413 CAN transceiver. Several boards can share one CAN cable. A PLC or PC talks to them by sending short CAN frames. The motor does not understand “CANopen” by itself — the firmware on the ESP32 does.

1.2 CAN is the wire. CANopen is the language

Think of three layers:

Layer	What it is	On this product
Physical CAN	Two wires (CAN_H / CAN_L), 11-bit IDs, up to 8 data bytes	CAN 2.0A, default 1 Mbps
CANopen (CiA 301)	Agreed meaning for those IDs: start node, read a setting, cyclic I/O	NMT, SDO, PDO, heartbeat, emergency
Drive profile (CiA 402)	Agreed meaning for a motor: enable, move, home, fault	pp / pv / hm subset

Without CANopen, a PLC only sees raw frames. With CANopen, the PLC can say “write object 0x607A (target position)” instead of packing bytes by hand.

1.3 What an EDS file is

An **Electronic Data Sheet** (CiA 306) is a text file that describes one device to a PLC configuration tool. It is not firmware and it does not move the motor. It is a catalogue:

- who made the device (vendor, product, revision)
- which bitrates it supports
- every setting and process value (the *object dictionary*)
- which values can ride on cyclic PDOs
- default CAN identifiers (COB-IDs)

When you import `GrafitoCANStepper.eds`, TwinCAT, Codesys, TIA Portal, or similar tools build a device tree from that catalogue. The PLC then uses SDO and PDO at runtime against the live node.

Three files are generated from the same object table in `canopen402.py`:

File	Standard	What it is
<code>GrafitoCANStepper.eds</code>	CiA 306 EDS	Device <i>template</i> . Import this to add the product to the PLC catalogue.
<code>GrafitoCANStepper_Node1.dcf</code>	CiA 306 DCF	One <i>instance</i> : node ID 1, 1 Mbps, filled COB-IDs, ParameterValue on every object. Use this when the tool asks for a DCF / “configured device”.
<code>object_dictionary.json / .csv</code>	Grafito export	The same object dictionary as a table. For spreadsheets, reviews, and tools that do not parse EDS.

Analogy

The EDS is the datasheet. The DCF is that datasheet filled in for one board on the bus. The object dictionary is the list of registers. SDO is “read/write one register when I ask.” PDO is “keep sending these registers every cycle.” NMT is the on/off switch for the whole node.

1.4 Where the files live

Item	Path
EDS (device template)	can_stepper/firmware/GrafitoCANStepper_C3_CANopen/GrafitoCANStepper.eds
DCF (node 1 instance)	can_stepper/firmware/GrafitoCANStepper_C3_CANopen/GrafitoCANStepper_Node1.dcf
Object dictionary	object_dictionary.json and object_dictionary.csv (same folder)
CANopen firmware	firmware/GrafitoCANStepper_C3_CANopen/GrafitoCANStepper_C3_CANopen.ino
Protocol stack in firmware	firmware/GrafitoCANStepper_C3_CANopen/canopen_stack.h
Host / test source of truth	can_stepper/canstepper/canopen402.py
USB bring-up script	can_stepper/tools/canopen_bringup.py

1.5 Identity numbers the PLC will show

Field	Value	Meaning
Vendor name	Grafito Innovations	Shown in the device catalogue
Vendor number	0x000005A3	Development placeholder. Register a real CiA vendor ID before a production EDS
Product name	Grafito CANStepper C3	
Product number	0x00004333	Object 0x1018:02
Revision	0x00020000	Firmware 2.0.0
Order code	GRAFITO-CANSTEPPER-C3	
Device type 0x1000	0x00040192	Stepper + CiA 402
Device name 0x1008	CANStepper	
Default node ID	1 (range 1-127)	Not the old GCSP 1-31 limit

1.6 Units you must memorise

CANopen drives use integers, not floating-point degrees on the wire.

Quantity	Unit	Conversion
Position (0x6064, 0x607A)	encoder counts	16384 counts = 360° = 1 motor revolution
Velocity (0x606C, 0x60FF, 0x6081)	counts / second	deg/s = counts/s × 360 / 16384
Acceleration (0x6083, 0x6084, 0x6085)	counts / s ²	same scale. Default 131072 ≈ 2880 °/s ²

Examples: $90^\circ = 4096$ counts. $10^\circ \approx 455$ counts. $720^\circ/\text{s} = 32768$ counts/s (the default profile velocity).

First practical step

Flash firmware 2.0, apply 24 V Vin (USB is data only), terminate the two physical ends of the CAN bus, set the PLC bitrate to 1 Mbps, import the EDS, set node ID 1, then read 0x1000 and 0x1018 before you try to move.

Part 2 — Intermediate: the EDS file and object dictionary

INTERMEDIATE

2.1 EDS is an INI file

Open `GrafitoCANStepper.eds` in any text editor. Sections look like `[FileInfo]`, `[DeviceInfo]`, `[1000]`, `[1018sub1]`. Comments start with `;`.

FileInfo

Metadata for the file itself: name, EDS version 4.0, author, dates. PLC tools record this; it does not change runtime behaviour.

DeviceInfo

Tells the tool which baud rates to offer and how many PDOs exist.

Key	This EDS	What you set on the PLC
BaudRate_125 / 250 / 500 / 1000	1 (supported)	Match object 0x200E. Factory default is 1 Mbps
BaudRate_10 / 20 / 50 / 800	0	Do not select these
SimpleBootUpSlave	1	Node boots to pre-operational and waits for NMT Start
NrOfRXPDO / NrOfTXPDO	2 / 2	RPDO1+2, TPDO1+2
LSS_Supported	0	No LSS. Change node ID via 0x2000 + save + reboot
Granularity	8	Mapping is byte-oriented

Object lists

Three inventories must list every index that later appears as a section:

- **MandatoryObjects** — 0x1000, 0x1001, 0x1018 (CiA 301 requires these)
- **OptionalObjects** — communication + CiA 402 objects
- **ManufacturerObjects** — Grafito-specific 0x2000–0x2019

If an index is in the file but missing from these lists, many importers reject the EDS.

DCF extras (not in the EDS)

The DCF is the EDS plus a commissioned instance. Extra sections:

- **DeviceComissioning** (CiA spelling) — `NodeID=1`, `Baudrate=1000` (kbit/s), `NetworkName=GrafitoCAN`
- **LastEDS** — points at `GrafitoCANStepper.eds`
- **ParameterValue** on every object — the live instance. `DefaultValue` stays the factory EDS default. For node 1 they usually match; for another node ID the COB-IDs and 0x2000 differ.

Regenerate a DCF for another node with `render_dcf(node_id=N)`. Do not hand-edit COB-IDs; they are 0x80+N, 0x180+N, 0x200+N, ...

2.2 Anatomy of one object

```
[6040]
ParameterName=Controlword
ObjectType=0x7          ; 0x7 = VAR (single value). 0x8 = ARRAY. 0x9 = RECORD
DataType=0x0006         ; UNSIGNED16
AccessType=rw           ; ro / rw / wo
DefaultValue=0
PDOMapping=1           ; 1 = may be mapped into a PDO
```

Records (identity, PDO communication) use [1018] plus [1018sub0] ... [1018sub4]. Subindex 0 is always “number of entries.”

2.3 Data types used in this EDS

EDS DataType	Name	Size	Used for
0x0002	INTEGER8	1	Modes, homing method
0x0003	INTEGER16	2	Quick-stop option
0x0004	INTEGER32	4	Position, velocity (signed)
0x0005	UNSIGNED8	1	Flags, currents, node ID
0x0006	UNSIGNED16	2	Controlword, statusword, heartbeat
0x0007	UNSIGNED32	4	IDs, profile limits, mappings
0x0008	REAL32	4	PID gains, bus voltage
0x0009	VISIBLE_STRING	variable	Name / versions (SDO segmented)

Multi-byte values are **little-endian**.

2.4 Communication objects (CiA 301)

Index	Name	Access	Role
0x1000	Device Type	ro	Must read 0x00040192
0x1001	Error Register	ro	Bit0 generic, bit1 current, bit3 temp, bit5 profile
0x1005	COB-ID SYNC	rw	Default 0x080. SYNC not required for event-driven PDOs
0x1008 / 09 / 0A	Name, HW, SW version	ro	CANStepper / C3 / 2.0.0
0x1014	COB-ID EMCY	ro	0x080 + node ID
0x1017	Producer heartbeat time	rw	Milliseconds. Default 1000. 0 = off
0x1018:01-04	Identity	ro	Vendor, product, revision, serial
0x1200	SDO server	ro	Rx 0x600+n, Tx 0x580+n
0x1400 / 1401	RPDO1/2 communication	rw type	COB-ID and transmission type 255
0x1600 / 1601	RPDO1/2 mapping	ro	Fixed mapping (see Part 4)
0x1800 / 1801	TPDO1/2 communication	rw type	Cyclic/event while operational
0x1A00 / 1A01	TPDO1/2 mapping	ro	Fixed mapping

2.5 Manufacturer objects (Grafito 0x2000 range)

These are not in the CiA 402 standard. Use SDO in pre-operational or operational. After changing node ID or bitrate, write 0x2008 = 1 (save) and reset the node.

Index	Name	Type	Default	Notes
0x2000	Node ID	U8	1	1-127. Save + reboot to apply COB-IDs
0x2001	Steps per revolution	U32	200	Motor full steps
0x2002	Microsteps	U32	16	Power of two, 1-256
0x2003	Run current %	U8	20	1-100 of TMC peak
0x2004	Hold current %	U8	5	0-100
0x2005	Stall threshold	U8	10	TMC SGTHRS
0x2006	Invert direction	U8	0	Set 1 if closed loop runs the wrong way
0x2007	Closed loop enable	U8	1	1 = encoder PID on position moves
0x2008	Save config	U8 wo	—	Write 1 to persist NVS
0x2009	Load defaults	U8 wo	—	Write 1; node ID kept
0x200A	StealthChop	U8	1	0 = SpreadCycle
0x200B	Standstill mode	U8	1	0 normal, 1 freewheel, 2 brake, 3 strong
0x200C	Endstop enable	U8	1	GPIO8 / HOME
0x200D	Endstop active high	U8	1	Pressed = GPIO8 HIGH
0x200E	CAN bitrate	U32	1000000	125000 / 250000 / 500000 / 1000000
0x200F-2011	PID Kp / Ki / Kd	R32	12 / 0.3 / 0.10	Closed-loop tracking
0x2012	PID tolerance	R32	0.35	Degrees
0x2013	Enable on boot	U8	0	Keep 0 so coils stay off until 402 enable
0x2014	Zero on boot	U8	0	
0x2015	Homing timeout	U32	30000	Milliseconds
0x2016	Firmware version	U16 ro	0x0200	Major in high byte
0x2017	Encoder OK	U8 ro	1	
0x2018	Endstop active	U8 ro	0	Logical HOME after polarity
0x2019	Bus voltage	R32 ro	24.0	Reported rail; C3 has no VBUS ADC

Invert direction (0x2006)

On the first live CANopen bench, a +10° command ran the opposite way and faulted. Many GCSP boards were stored with INVERT_DIR=1. CANopen NVS is a new namespace, so invert defaults to 0. If the motor runs away from the target, set 0x2006 = 1, save, and retry a small move.

Part 3 — Intermediate: CiA 402 drive profile INTERMEDIATE

3.1 Why a PLC will not move from “enable bit = 1”

CiA 402 uses a **finite state machine**. The PLC walks the drive through named states by writing the controlword (0x6040). The drive reports the state in the statusword (0x6041). Skipping steps is ignored.

State	Statusword bits 0-6	Hex (plus remote 0x200)	Coils / motion
Switch on disabled	xxxx 0100 0000	0x0240	Off after boot
Ready to switch on	xxxx 0010 0001	0x0231	Still off
Switched on	xxxx 0010 0011	0x0233	Power stage ready
Operation enabled	xxxx 0010 0111	0x0237	Motion allowed
Quick stop active	xxxx 0000 0111	0x0207	Stopping
Fault	xxxx 0000 1000	0x0208	Disabled; need fault reset

Bit 4 (voltage enabled, 0x0010) and bit 9 (remote, 0x0200) are also set in the ready / enabled states on this firmware.

3.2 Controlword commands (0x6040)

The command is encoded in bits 0–3 and 7. Other bits (new setpoint, relative, halt) are used only in Operation enabled.

Command	Write value (typical)	Bits that matter	From → to
Shutdown	0x0006	xxxx x110	SOD → Ready; OE → Ready
Switch on	0x0007	xxxx 0111	Ready → Switched on; also “disable operation” from OE
Enable operation	0x000F	xxxx 1111	Switched on → Operation enabled
Disable voltage	0x0000	bit1 = 0	Back to Switch on disabled
Quick stop	0x0002	xxxx x01x	OE → Quick stop
Fault reset	0x0080	bit7 rising edge	Fault → Switch on disabled
New setpoint (pp/hm)	0x001F	0x000F + bit4	Starts a position or home
Halt	0x010F	bit8	Stop, target reached
Relative pp	0x005F	bit6 + bit4 + 0x0F	Target is a delta

Memorise this enable sequence

Write 0x6040 =

6

, then

7

, then

15

(0x0F). Read 0x6041 after each step until bits 0–6 equal 0x21, 0x23, then 0x27. Only then write a target.

3.3 Statusword extra bits

Bit	Mask	Meaning on this drive
10	0x0400	Target reached (move finished or halt complete)
12	0x1000	pp: set-point acknowledge / hm: homing attained
13	0x2000	Following error (if raised)

3.4 Modes of operation (0x6060 / 0x6061)

Value	Name	How you command it
1	Profile position (pp)	Write 0x607A, then set controlword new-setpoint bit
3	Profile velocity (pv)	Write 0x60FF while enabled; 0 = stop
6	Homing (hm)	Write 0x6098 method, then new-setpoint bit
0	No mode	Idle

0x6502 (supported modes) = 0x00000025 (bit 0 = pp, bit 2 = pv, bit 5 = hm). Interpolated position, cyclic synchronous, and torque modes are **not** implemented.

3.5 Profile position recipe

1. Optional: set 0x6081 (cruise) and 0x6083 (accel) in counts/s and counts/s².
2. Write 0x6060 = 1.
3. Walk 6 → 7 → 15 on 0x6040.
4. Write 0x607A = target counts (absolute unless controlword bit 6 is set).
5. Write 0x6040 = 0x001F (enable + new setpoint).
6. Write 0x6040 = 0x000F (drop bit 4; leave enabled). Status bit 12 acks the edge.
7. Wait until 0x6041 bit 10 (target reached) or 0x6064 is inside tolerance.

3.6 Homing methods (0x6098)

Value	Behaviour
35 or 0	Define current shaft position as zero (no motion)
17	Seek HOME endstop, negative direction
18	Seek HOME endstop, positive direction
−1	StallGuard seek, negative (manufacturer)
−2	StallGuard seek, positive (manufacturer)

Speeds: 0x6099:01 search, 0x6099:02 zero-window. Timeout: 0x2015. GPIO8 is a strapping pin — it must be HIGH at power-on or the ESP32-C3 may not boot.

3.7 Faults (0x603F and EMCY)

Code	Name	Typical cause
0x0000	None	
0x2310	Short / overcurrent	TMC S2G or low-side short
0x4310	Over-temperature	TMC OT shutdown
0x7305	Encoder	SSI loss longer than 25 ms
0x7122	Blocked / no progress	Closed loop not reducing error
0xFF01	Homing failed	Timeout or illegal method

An EMCY frame is sent on 0x080 + node ID: bytes 0–1 = error code, byte 2 = error register. Recover with controlword fault reset (0x0080), then repeat 6 → 7 → 15.

Part 4 — Advanced: PDOs, COB-IDs, SDO and bus timing

ADVANCED

4.1 COB-ID map for node N

Standard 11-bit identifiers. Defaults below are for **N = 1** as stored in the EDS. Live firmware adds the actual node ID.

Service	Formula	Node 1	Direction
NMT	0x000	0x000	Master → all / one
SYNC	0x080	0x080	Master → slaves
EMCY	0x080 + N	0x081	Node → master
TPDO1	0x180 + N	0x181	Status + position
RPDO1	0x200 + N	0x201	Controlword + target position
TPDO2	0x280 + N	0x281	Status + velocity
RPDO2	0x300 + N	0x301	Controlword + target velocity
TSDO (reply)	0x580 + N	0x581	Node → master
RSDO (request)	0x600 + N	0x601	Master → node
Heartbeat	0x700 + N	0x701	Node → master (1 byte NMT state)

Heartbeat payload: 0x00 boot-up, 0x04 stopped, 0x05 operational, 0x7F pre-operational.

NMT payload (ID 0x000): byte0 = command (0x01 start, 0x02 stop, 0x80 pre-op, 0x81 reset node, 0x82 reset comm), byte1 = node ID or 0 for all.

Why GCSP and CANopen cannot share a bus

GCSP uses CAN ID = (node_id << 6) | msg_id. That range collides with CANopen SYNC (0x080), EMCY, TPDOs and heartbeats. Flash one protocol per network.

4.2 PDO mapping encoding

Each mapping entry is a 32-bit value: bits 31-16 = index, 15-8 = subindex, 7-0 = bit length.

Object	Stored value	Decoded
0x1600:01	0x60400010	Controlword 0x6040:00, 16 bits
0x1600:02	0x607A0020	Target position 0x607A:00, 32 bits
0x1601:01	0x60400010	Controlword, 16 bits
0x1601:02	0x60FF0020	Target velocity, 32 bits
0x1A00:01	0x60410010	Statusword, 16 bits
0x1A00:02	0x60640020	Position actual, 32 bits
0x1A01:01	0x60410010	Statusword, 16 bits
0x1A01:02	0x606C0020	Velocity actual, 32 bits

RPDO1 payload (8 bytes): U16 controlword | I32 target_pos | U16 pad. RPDO2 replaces the I32 with target velocity. TPDOs use the same layout with statusword + actual value.

Transmission type **255** = event / manufacturer (async). Firmware also emits TPDOs about every 20 ms while NMT is operational. RPDOs are ignored unless the node is operational. Mapping is **fixed** in firmware; SDO writes to 0x1600/0x1A00 are read-only.

4.3 SDO protocol the EDS implies

All configuration objects are reached with expedited SDO except visible strings (0x1008/09/0A), which use segmented upload.

```
Request ID 0x600+N
  cmd, index_lo, index_hi, sub, d0, d1, d2, d3

Upload request      cmd = 0x40
Expedited download 4 B cmd = 0x23  (3 B 0x27, 2 B 0x2B, 1 B 0x2F)
Download OK         cmd = 0x60
Expedited upload 4 B cmd = 0x43
Initiate segmented  cmd = 0x41 + size
Abort               cmd = 0x80 + abort code
```

Common abort codes: 0x06020000 no object, 0x06090011 no subindex, 0x06010002 write to RO, 0x06090030 value rejected (bad bitrate, non-power-of-two microsteps, illegal mode).

4.4 NMT vs 402 — two state machines

Beginners mix these up. They are independent:

- **NMT** (network): boot → pre-op (SDO yes, PDO no) → operational (SDO + PDO) → stopped (almost mute) → reset.
- **402** (drive): switch-on-disabled → ... → operation enabled / fault.

You can SDO-enable the 402 machine while still in NMT pre-op (useful for commissioning).
Cyclic RPDO motion only works after NMT Start (0x01).

4.5 Editing the EDS (when you are allowed)

Safe edits: comments, ParameterName text, FileRevision, description. Dangerous edits: removing an object the firmware still implements, changing DataType, or advertising a PDO map the stack does not use. After any object change, regenerate EDS, DCF and the object-dictionary exports together with `write_canopen_descriptions()` so the Python tests, PLC files and firmware stay aligned.

DummyUsage Dummy0002-0007 = 1 lets tools pad PDO slots. Leave as shipped.

Part 5 — Advanced: PLC import and first-motion recipes

ADVANCED

5.1 Wiring and tool setup (all brands)

1. Flash GrafitoCANStepper_C3_CANopen (Arduino FQBN esp32:esp32:esp32c3:CDCC0nBoot=cdc).
2. Apply Vin 5-24 V (typically 24 V). USB-C does not power the MCU for reliable programming or motion.
3. CAN: twisted pair, 120 Ω only on the two physical ends. On-board 0 Ω termination jumper is fitted by default — remove it on mid-chain boards.
4. PLC CAN adapter: **1 Mbps**, 11-bit, no 29-bit, same ground reference.
5. Import GrafitoCANStepper.eds as the device type. If the tool wants a configured node / DCF, import GrafitoCANStepper_Node1.dcf (node 1, 1 Mbps). For another node ID, regenerate the DCF — do not reuse node 1 COB-IDs.
6. Set slave node ID to the value stored on the board (default 1).
7. Scan / go online. Read 0x1018. If vendor/product mismatch, you have the wrong EDS or the wrong firmware.

5.2 Codesys (CAN bus + CiA 405 / 3S CANopen)

1. Add a CANopen Manager under the CAN interface. Bitrate 1000 kbit/s.
2. Right-click manager → Add Device → Install vendor EDS → select GrafitoCANStepper.eds.
3. Add “Grafito CANStepper C3” as a slave. Node ID 1. Heartbeat consumer 1.5× 1000 ms is a good watchdog.
4. Map RPDO1 (controlword + target pos) and TPDO1 (statusword + position) into your IEC program as integers.
5. Startup SDO list (optional): 0x2003 run current, 0x2006 invert, 0x200C endstop, 0x6081 speed, 0x6060 = 1.
6. NMT Start is usually sent by the manager automatically.

5.3 TwinCAT 3 (Beckhoff)

1. I/O → Devices → CAN → right-click → Import EDS.
2. Insert the box, set Node ID and 1 Mbaud on the adapter (EL6751, FC510x, CX CAN, ...).
3. Process data tab should show the four PDOs. Create PLC variables linked to 0x6040, 0x6041, 0x607A, 0x6064.
4. Free-run or start the NC/PLC. Use a small state machine FB: shutdown / switch on / enable / new setpoint.

5.4 TIA Portal / other Siemens CAN (via gateway)

S7-1200/1500 do not speak CiA 402 natively on a random CAN dongle. Typical paths: a dedicated CANopen master (HMS, Ixxat, Siemens CM CANopen) that imports the EDS, or a Linux SoftPLC (Codesys) as gateway. Import the same EDS on that master. Do not expect a raw “CAN_SEND” FB to honour the 402 state machine unless you write it yourself.

5.5 IEC-style enable and jog (pseudocode)

```
VAR
  cw AT %QW : UINT;      (* 0x6040 via RPD01 *)
  sw AT %IW : UINT;      (* 0x6041 via TPD01 *)
  tgt AT %QD : DINT;     (* 0x607A *)
  pos AT %ID : DINT;     (* 0x6064 *)
  step : INT;
END_VAR

CASE step OF
  0: cw := 16#0006; IF (sw AND 16#006F) = 16#0021 THEN step := 1; END_IF
  1: cw := 16#0007; IF (sw AND 16#006F) = 16#0023 THEN step := 2; END_IF
  2: cw := 16#000F; IF (sw AND 16#006F) = 16#0027 THEN step := 3; END_IF
  3: tgt := pos + 455;   (* +10 degrees *)
    cw := 16#001F;      (* new set-point *)
    step := 4;
  4: cw := 16#000F;      (* drop bit 4 *)
    IF (sw AND 16#0400) <> 0 THEN step := 5; END_IF
  5: (* dwell / next move *)
END_CASE

IF (sw AND 16#0008) <> 0 THEN (* fault *)
  cw := 16#0080; step := 0;
END_IF
```

5.6 USB serial bridge (no PLC, same objects)

Firmware still prints and accepts hex lines: <CAN_ID hex> <0|1> <payload hex>. Opening the USB-JTAG port resets the C3 — wait for # CAN started and heartbeat 0x7F.

```
python3 tools/canopen_bringup.py /dev/ttyACM0 --node 1
# identity only; add --move-deg only after invert is verified
```

Example SDO read of device type on node 1:

```
601 0 4000100000000000
# expect 581 0 4300100092010400 → 0x00040192
```

5.7 Recommended first-commission checklist

1. Read 0x1000, 0x1018, 0x2016, 0x2017 (encoder OK).
2. If 0x2018 (endstop) is 1 and you are not at a real home, write 0x200C = 0 for the first jog.
3. Set 0x2003 to a modest current (30) and 0x6081 to ~2275 (~50 °/s).
4. Enable 402. Confirm 0x6041 = 0x0237. Coils should hold; shaft should not run.
5. Command +455 counts. If the encoder moves the other way, abort, set 0x2006 = 1, save, reset, retry.
6. Only then raise speed/current and switch to RPDO cyclic control.

Part 6 — Reference tables and troubleshooting **ADVANCED**

6.1 Full CiA 402 object list in this EDS

Index	Name	Type	Access	PDO	Default
0x603F	Error Code	U16	ro	no	0
0x6040	Controlword	U16	rw	yes	0
0x6041	Statusword	U16	ro	yes	0x0240
0x605A	Quick Stop Option Code	I16	rw	no	2
0x6060	Modes of Operation	I8	rw	yes	1
0x6061	Modes of Operation Display	I8	ro	yes	1
0x6064	Position Actual Value	I32	ro	yes	0
0x606C	Velocity Actual Value	I32	ro	yes	0
0x607A	Target Position	I32	rw	yes	0
0x6081	Profile Velocity	U32	rw	no	32768
0x6083	Profile Acceleration	U32	rw	no	131072
0x6084	Profile Deceleration	U32	rw	no	131072
0x6085	Quick Stop Deceleration	U32	rw	no	262144
0x6098	Homing Method	I8	rw	no	35
0x6099:1/2	Homing speeds	U32	rw	no	2275 / 455
0x609A	Homing Acceleration	U32	rw	no	131072
0x60FF	Target Velocity	I32	rw	yes	0
0x6502	Supported Drive Modes	U32	ro	no	0x25

6.2 What this EDS does *not* claim

- No CiA 402 interpolated position (mode 7), cyclic synchronous position/velocity, or torque mode.
- No safety (STO, CiA 402 Safety / FSoE).
- No LSS node-ID assignment over CAN.
- No dynamic PDO remapping.
- No official CiA vendor ID yet (0x5A3 is a placeholder).
- Leader/follower (GCSP FOLLOW) is not exposed in CANopen.

6.3 Troubleshooting

Symptom	Likely cause	What to do
EDS import fails	Wrong encoding, missing object list	Use the generated file as-is; ASCII only
Scan finds nothing	Wrong bitrate, termination, GCSP still flashed	Confirm boot line "CANopen fw 2.0", 1 Mbps, 120 Ω
SDO abort 0x06020000	Tool writing an object not in this subset	Disable that object in the PLC device
PDOs silent	NMT still pre-op	Send NMT Start 0x01
Enable does nothing	Skipped 6→7→15	Read 0x6041 after each write
Move runs away / faults 0x7122	Invert or encoder magnet	0x2006 = 1; check 0x2017; lower 0x6081
Node dead after power-up	GPIO8 held LOW at reset	HOME must be HIGH at boot (strapping)
USB open reboots the chip	USB-JTAG DTR/RTS	Hold DTR/RTS off; wait for heartbeat 0x7F
Old Python library fails	GCSP IDs vs CANopen IDs	Use canopen402 / a PLC, not canstepper GCSP

6.4 Regenerating the EDS, DCF and object dictionary

```
cd can_stepper
python3 -c "from canstepper.canopen402 import write_canopen_descriptions; print(write_canopen_de
python3 -m pytest tests/test_canopen.py
```

The pytest suite checks that every object in the Python dictionary appears in the EDS and DCF, that the DCF has DeviceComissioning + ParameterValue, that the JSON/CSV catalogues match, that identity constants match the firmware header, and that a virtual slave performs NMT, SDO, 402 enable, PDOs and homing.

6.5 Learning path (use this document in order)

1. Part 1 until you can explain EDS vs firmware vs CAN wire.
2. Flash 2.0 and read 0x1000 / 0x1018 (USB script or PLC scan).
3. Part 3: perform 6 → 7 → 15 and watch 0x6041.
4. One 10° move with low 0x6081 after invert is confirmed.
5. Part 4: switch the same move onto RPDO1 / TPDO1.
6. Part 5: import the shipped DCF (or regenerate for another node ID) and lock startup SDOs on the customer PLC.

Document generated for Grafito CANStepper C3 firmware 2.0, GrafitoCANStepper.eds (EDS 4.0) and GrafitoCANStepper_Node1.dcf.

Companion files: object_dictionary.json / .csv, canopen_stack.h, canopen402.py, tools/canopen_bringup.py.

Standards referenced: CiA 301 (application layer), CiA 306 (EDS/DCF), CiA 402 (drives). Grafito implements a subset, not a full certified stack.